VibeBench — architecture and method

Measuring SOLID conformance of autonomous coding agents on real merged pull requests

1. End-to-end pipeline (engineering view)

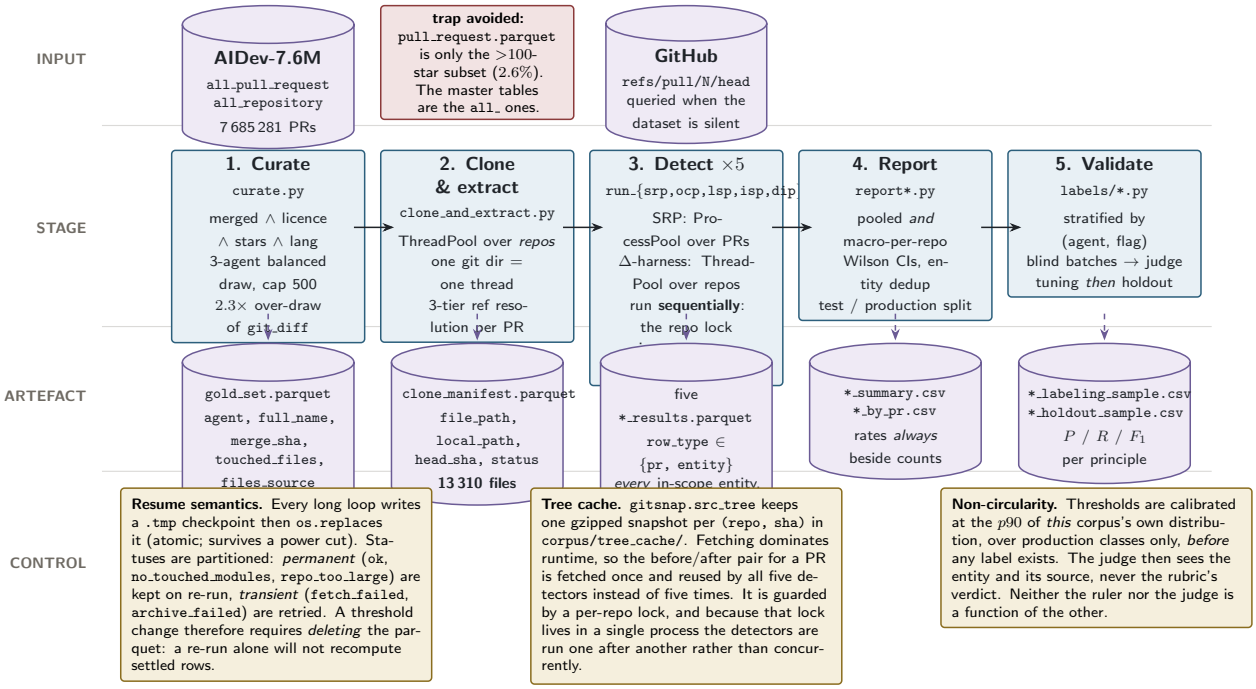


Figure 1: End-to-end pipeline, engineering view. Read top to bottom: what enters the system, what transforms it, what is written to disk, and the invariants that hold across the whole run. Each stage consumes only the previous stage's artefact, so any stage can be re-run without rebuilding the ones before it. The three control blocks are what make a multi-day run reproducible — crash-safe resumption, a cache that makes the dominant cost payable once, and an ordering that keeps threshold calibration independent of labelling.

2. Corpus selection

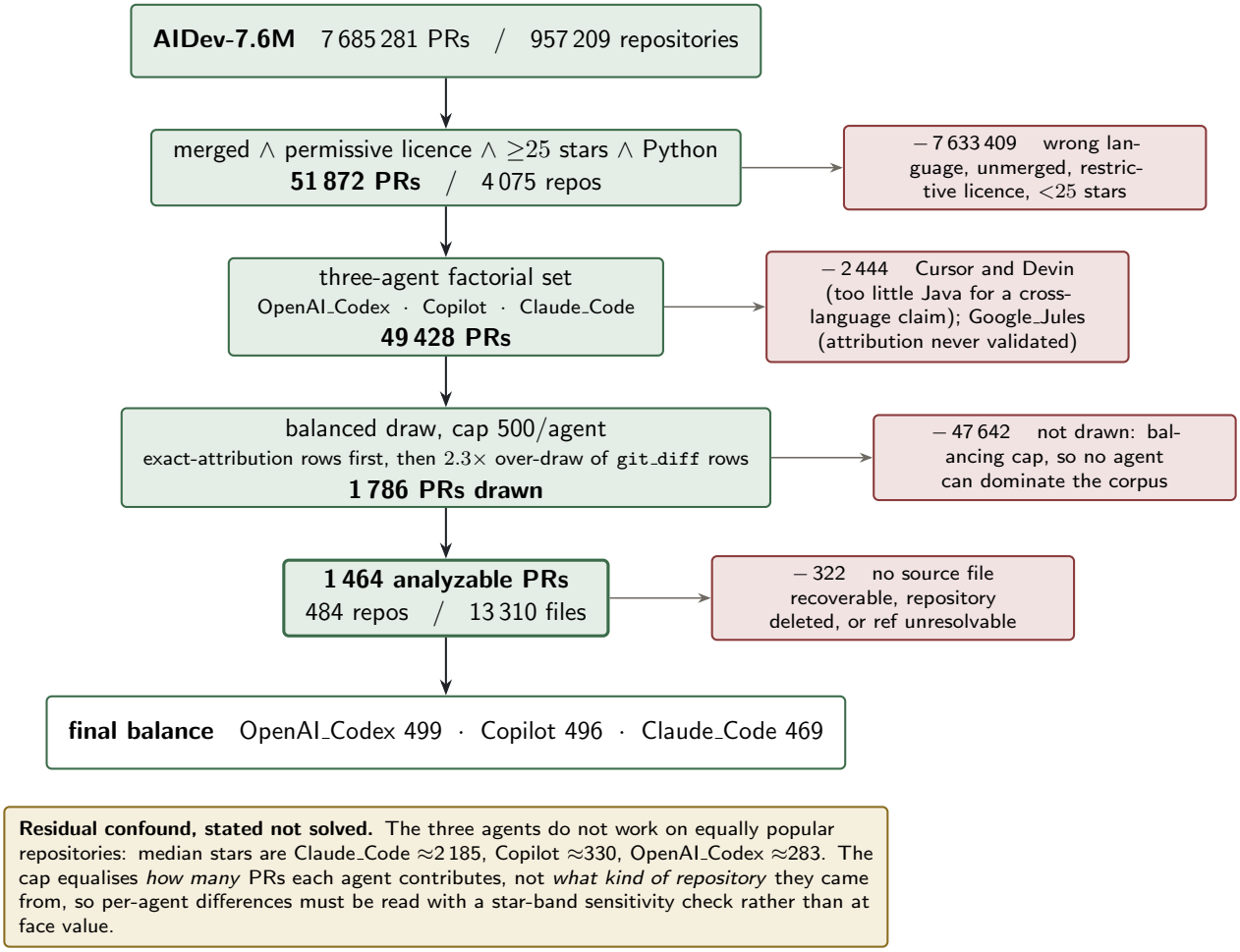


Figure 2: Corpus selection funnel. Each stage reports what survives (left) and what is removed and why (right), so the sample is reconstructible from the public dataset. The cap equalises agent contribution: uncapped, one agent supplies most of the corpus and every per-agent rate is dominated by that agent's own repository mix rather than by its behaviour.

3. What a "tree" is, and how a PR becomes a measurement

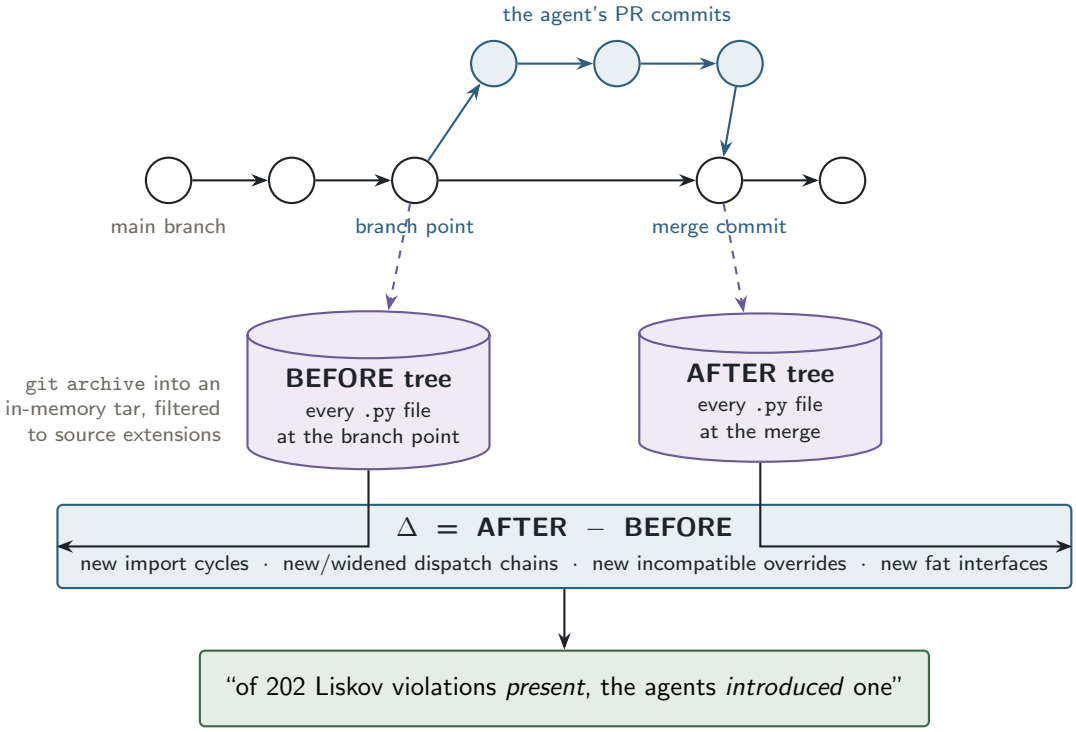
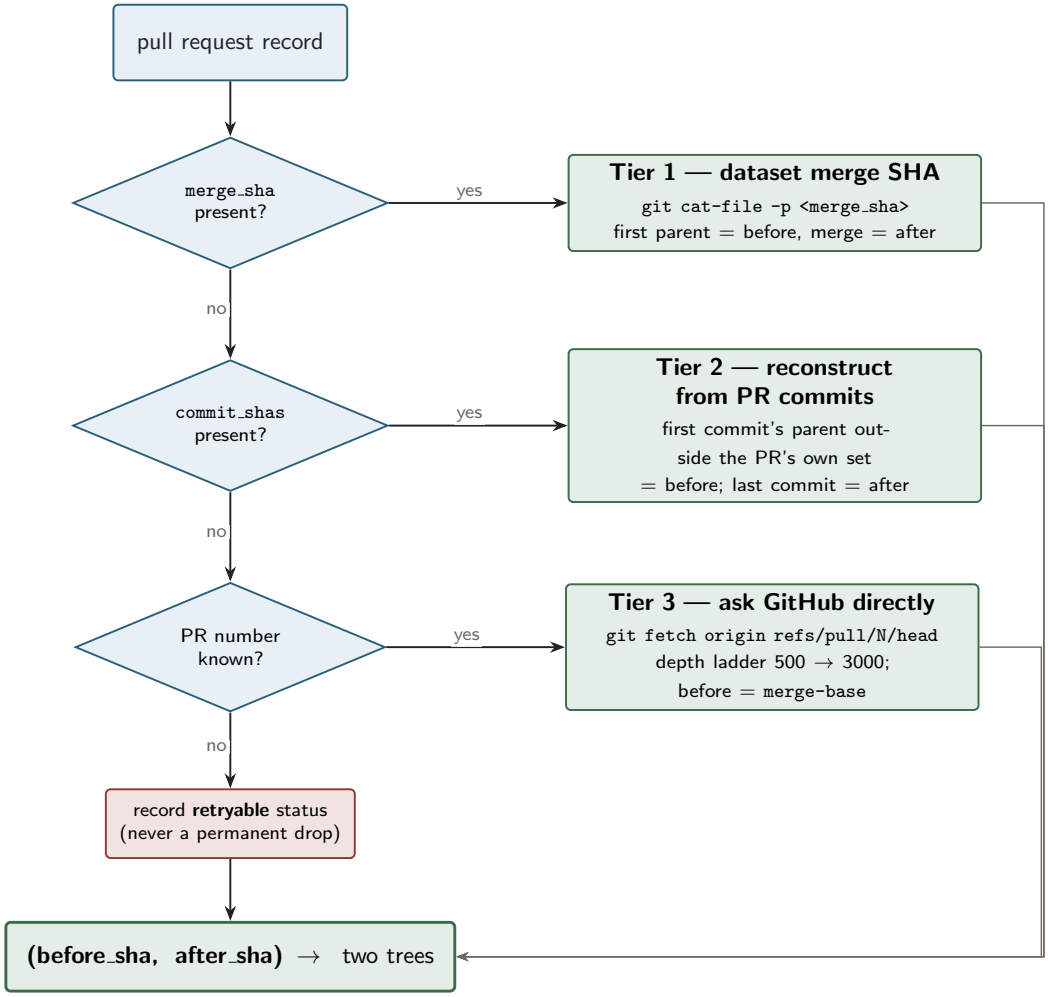


Figure 3: The delta method. A *tree* is a snapshot of every source file at one commit. Two are taken per pull request — at its branch point and at its merge — and the principles are evaluated on the difference between them. The branch point is located with `merge-base` against the default branch, *not* as the merge commit's first parent: for a multi-commit pull request the parent is the PR's own penultimate commit, which would capture only its final commit and silently understate everything the agent wrote.

4. Resolving a pull request to a commit pair



Why Tiers 2–3 exist, and why omitting them is a bias and not just a loss. `AIDev-7.6M` publishes per-PR commit tables for only **2.6%** of its pull requests (197 330 of 7 685 281) — the same high-popularity subset throughout. Coverage is therefore *correlated with repository popularity*, and popularity is not evenly distributed across agents. Measured on this corpus: before Tier 3, failed extractions had a median of 93 stars against 455 for survivors, shifting one agent's corpus **+85%** toward popular repositories while another's did not move at all. After Tier 3 that shift fell to +34%, in line with the other agents. Tier 3 requires nothing from the dataset: it asks GitHub for the pull request by number.

Figure 4: Three-tier commit resolution. The ladder is tried in order and stops at the first tier that resolves. A pull request that reaches the bottom is recorded with a *retryable* status rather than discarded, so a transient network failure never silently removes a PR from the corpus — only a permanent status does, and those are enumerated.

5. Units of analysis

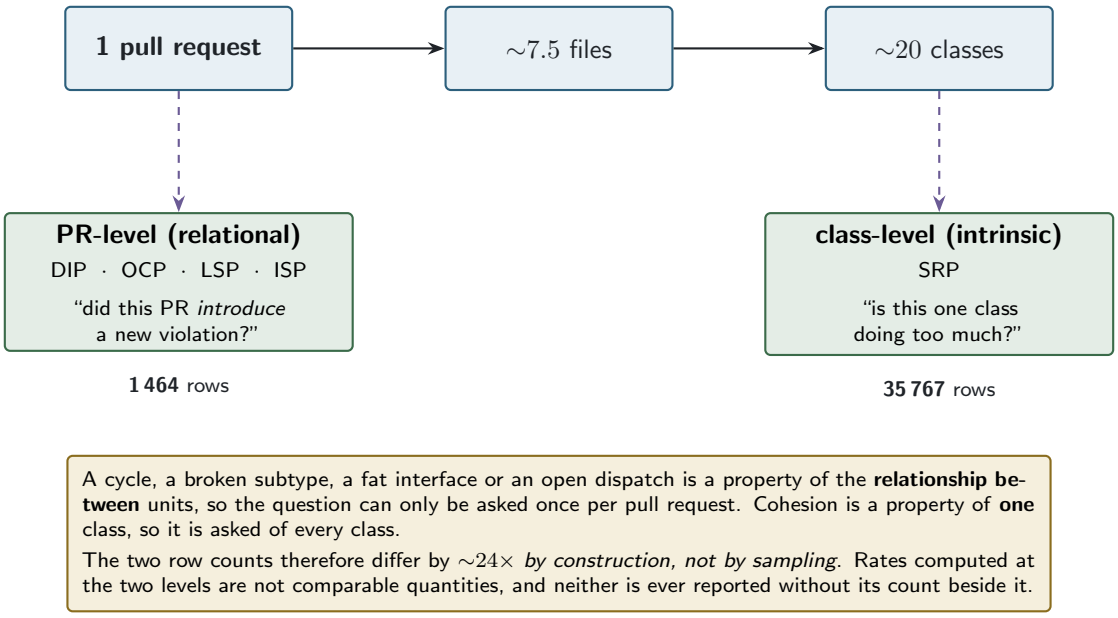
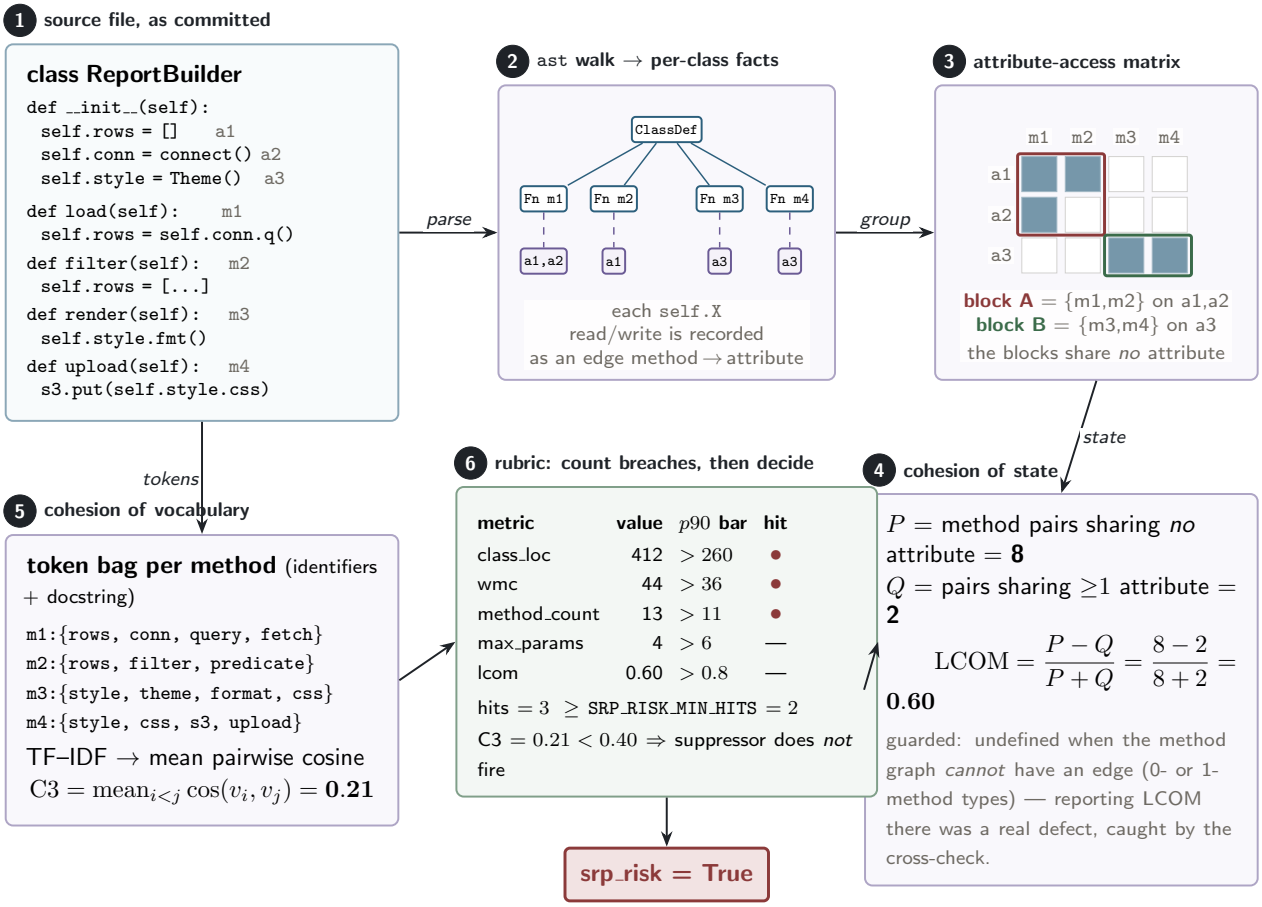


Figure 5: Why the principles produce very different row counts. The unit of analysis follows from what each principle asserts, not from a design preference. This is also why four of the five principles show single-digit PR-level flag counts: introducing both halves of a cross-unit smell inside one pull request is rare, which is itself the study's central finding rather than a shortfall of the instrument.

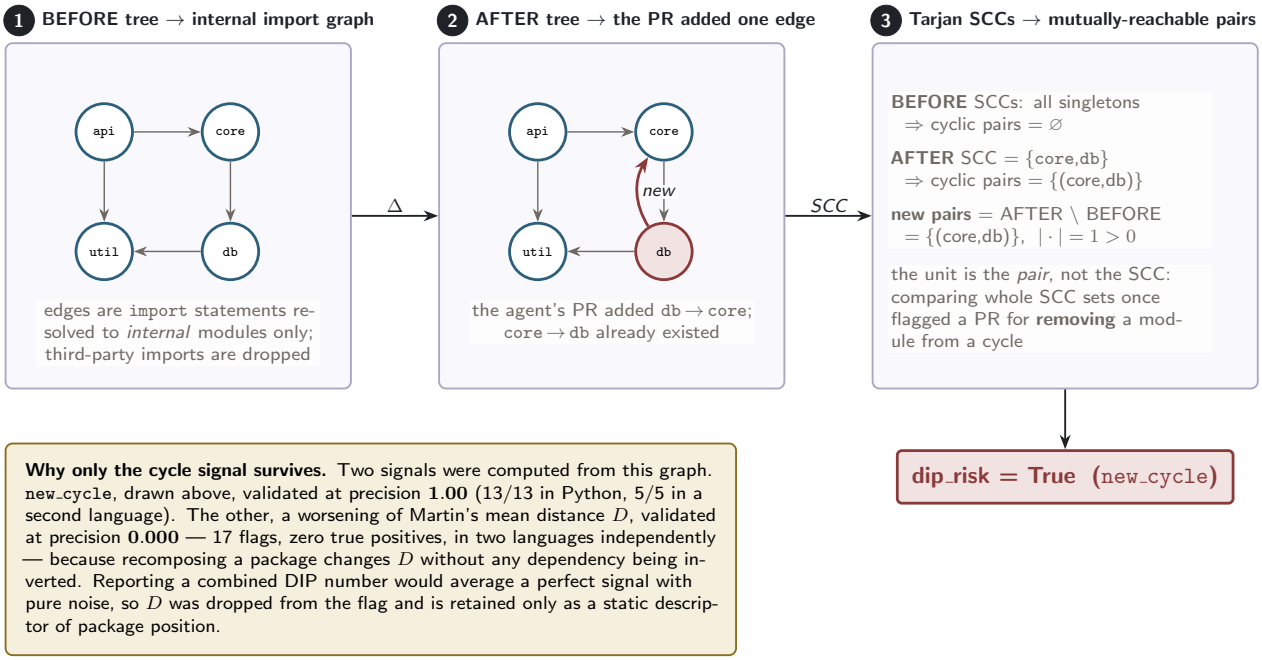
6. SRP detector, worked end to end



What this procedure can and cannot establish. Every quantity is *structural*: computed from the parse tree, without executing anything and without reading intent. Step ❹ is the entire basis of the cohesion claim — `load/filter` and `render/upload` touch disjoint state, which is the shape of a class with two reasons to change. But a framework base class whose members are mandated by an interface produces the *same* matrix, and that is the single largest source of false positives. At the shipped operating point the rubric reaches $R = 1.00$ (it misses essentially no real god class) at $P = 0.33$ (two of three flags are large-but-cohesive). The $p90$ bars are calibrated on this corpus's own distribution *before* any label exists, so the ruler is a property of the population, not of the answer.

Figure 6: SRP detector, worked end to end on a single class. Boxes shaded blue are inputs, purple are intermediate computations, green is the decision.

7. DIP detector, worked end to end



Why only the cycle signal survives. Two signals were computed from this graph. `new_cycle`, drawn above, validated at precision **1.00** (13/13 in Python, 5/5 in a second language). The other, a worsening of Martin's mean distance D, validated at precision **0.000** — 17 flags, zero true positives, in two languages independently — because recomposing a package changes D without any dependency being inverted. Reporting a combined DIP number would average a perfect signal with pure noise, so D was dropped from the flag and is retained only as a static descriptor of package position.

Figure 7: DIP detector: the same graph before and after, and the pair that appeared. Circles are internal modules, arrows are resolved imports. Only edges between modules of the repository itself are kept, so a dependency on an external library is invisible by construction.

8. OCP detector, worked end to end

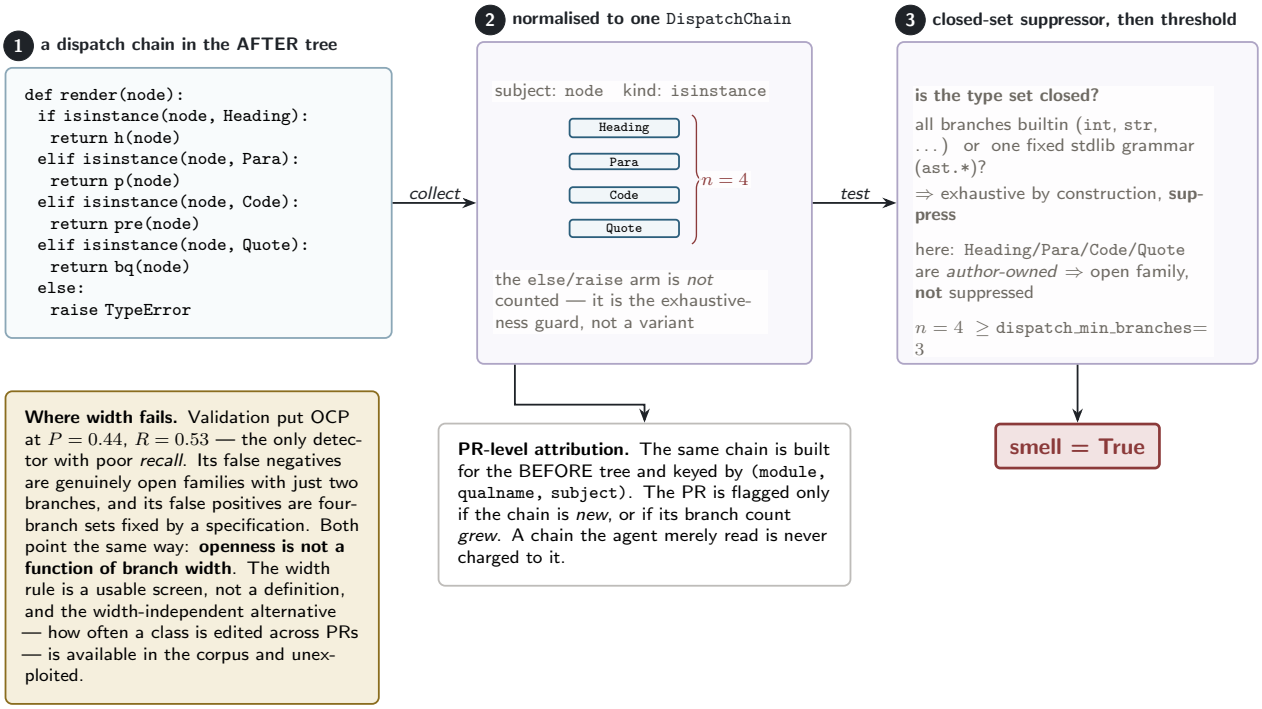


Figure 8: OCP detector: from an `isinstance` ladder to a flag. The chain is the unit of analysis; the branch count is the measurement; the closed-set test is the one deterministic correction that survived validation.

9. LSP detector, worked end to end

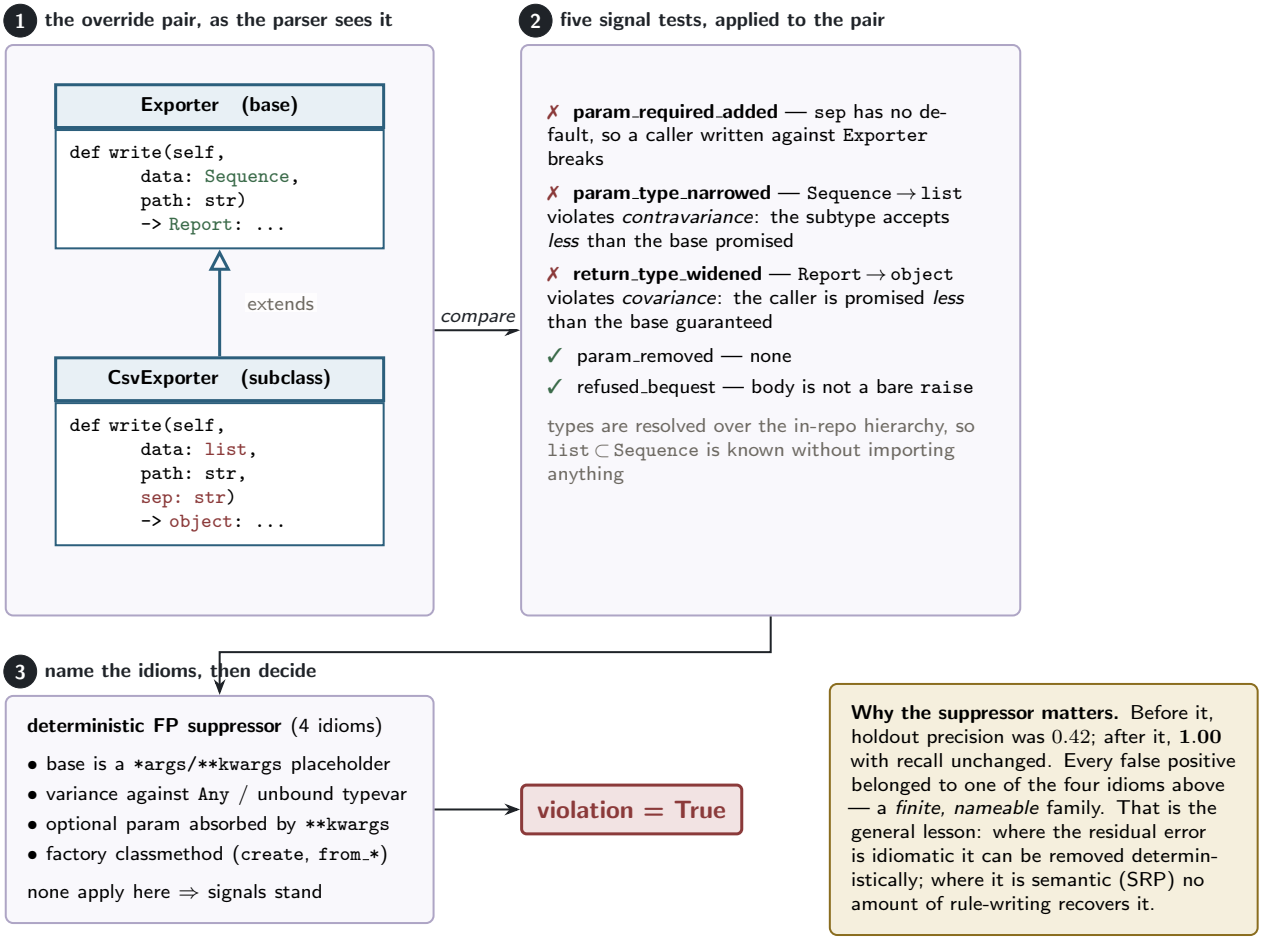


Figure 9: LSP detector: one override pair, five tests. The rule replicates what a type checker would report, on the AST, because 147 repositories cannot be made importable. Which signals are even *expressible* is language-dependent: `javac` makes three of these impossible in code that compiles, so the Java signal set keeps only what a compiler cannot check.

10. ISP detector, worked end to end

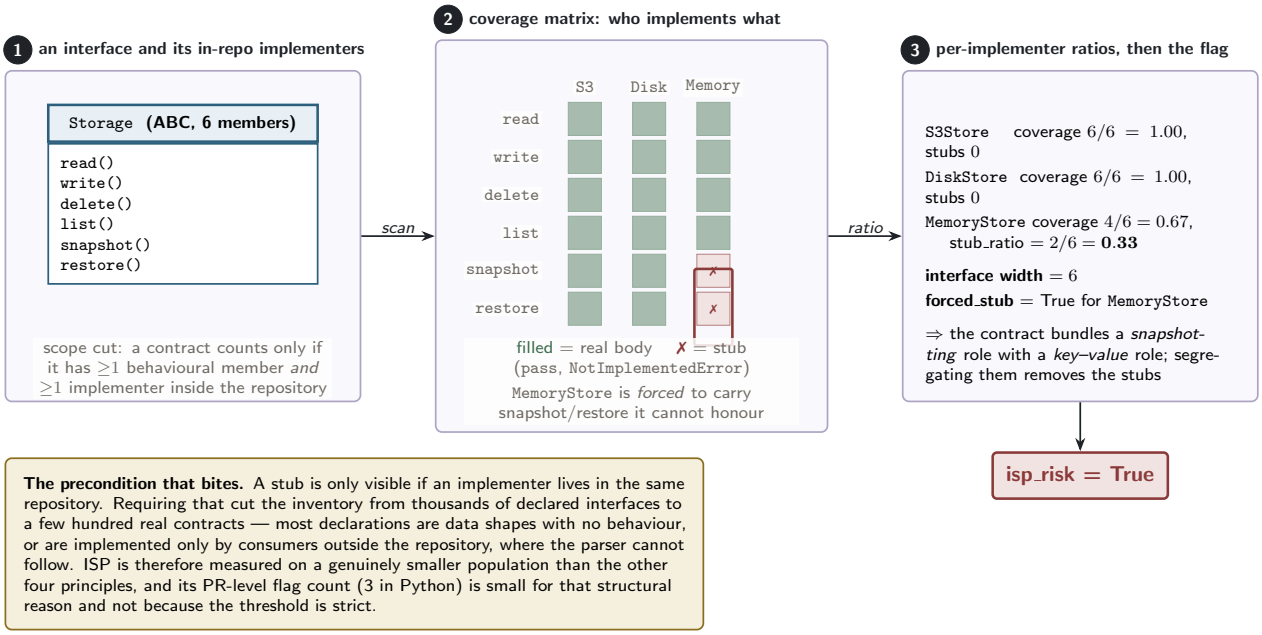


Figure 10: ISP detector: interface width against what implementers actually use. The matrix is the measurement; a column with stubbed cells is a client paying for members it does not need.

11. The proxies: what is asserted vs. what is computed

Principle (Martin)	Structural assertion	Computed proxy	What the proxy cannot see
SRP one reason to change	a class serving one concern is small, cohesive, and its methods share state	LOC >260, WMC >36, methods >11, params >6, LCOM >0.8, C3 cohesion; ≥ 2 hits $\Rightarrow$ flag	whether two concerns are genuinely *independent*; a framework-shaped class looks identical to a god class
OCP open to extension, closed to modification	an open variant family is dispatched on explicitly, so adding a variant edits existing code	`isinstance/type()/match` chain width; ≥ 3 type or ≥ 6 value branches; all-builtin sets suppressed	whether the set is *closed by specification*; a 2-branch author-owned family is open yet never flagged
LSP subtypes substitutable for their base	an unsubstitutable override changes the signature, the variance, or refuses inherited behaviour	override$\rightarrow$ancestor pairs: param added/removed, param type narrowed, return widened, refused bequest	a semantic contract break with an identical signature (returning empty where the base raises)
ISP no client forced to depend on what it does not use	a fat interface shows up as implementers stubbing members they do not need	ABC/Protocol width, implementer coverage, stub ratio, forced-stub count over in-repo implementers	implementers outside the repository; a wide but genuinely single-topic service contract
DIP depend on abstractions, not concretions	concrete-on-concrete dependence produces import cycles and pulls packages off the main sequence	Tarjan SCCs $\rightarrow$ newly mutually-reachable module *pairs*; Martin C_e, C_a, I, A, D per package	dependence on a concrete class that creates *no* cycle — the common case, and invisible here

Figure 11: Each principle and the structural proxy that stands in for it. SOLID is stated in terms of *reasons to change* and *substitutability* — semantic properties no parser can read directly. Every entry in column three is therefore a structural stand-in for the claim in column two, and column four names exactly what that substitution gives up. The proxies are not the principles; the next figure measures how far apart they actually are.

12. How far is each proxy from its principle?

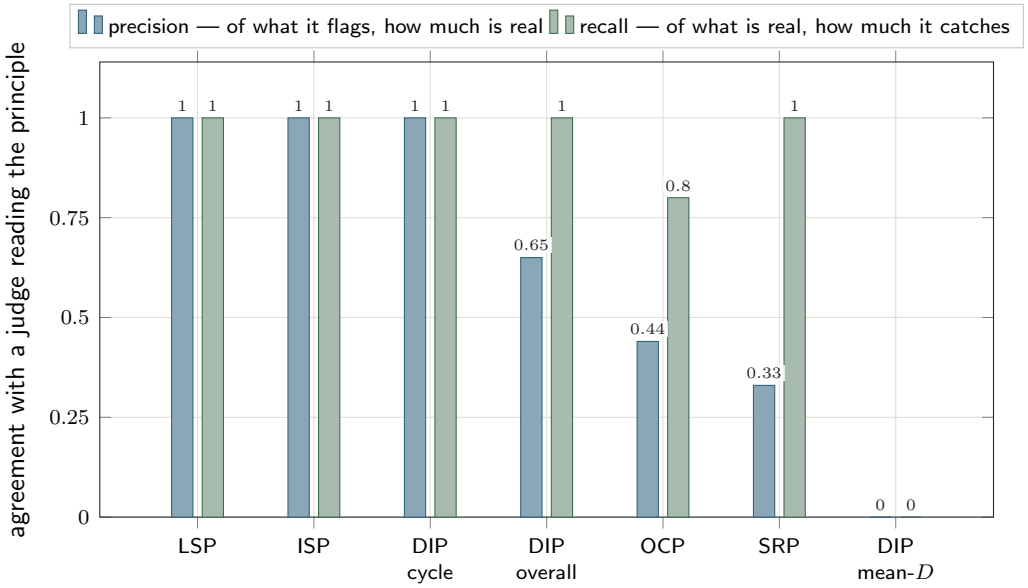


Figure 12: Proxy fidelity is a property of the principle, not of the effort spent. Python holdout figures from the completed validation round. The ordering is itself the result. Where a principle's definition is already *structural*, the proxy very nearly is the principle: LSP, ISP and DIP's cycle signal all reach precision 1.00. Where the definition is *semantic*, structure bounds recall but cannot recover precision — SRP catches essentially every real god class (R=1.00) while two of every three flags are large-but-cohesive classes. OCP sits between, and its errors explain why: the false positives are specification-closed sets and the false negatives are narrow author-owned ones, which together say that openness is not a function of branch width. The two DIP bars are the sharpest evidence available — *one principle, two proxies, precision* 1.00 *and* 0.00. Proxy fidelity must therefore be measured per signal, never assumed per principle.

13. Martin's abstractness–instability plane

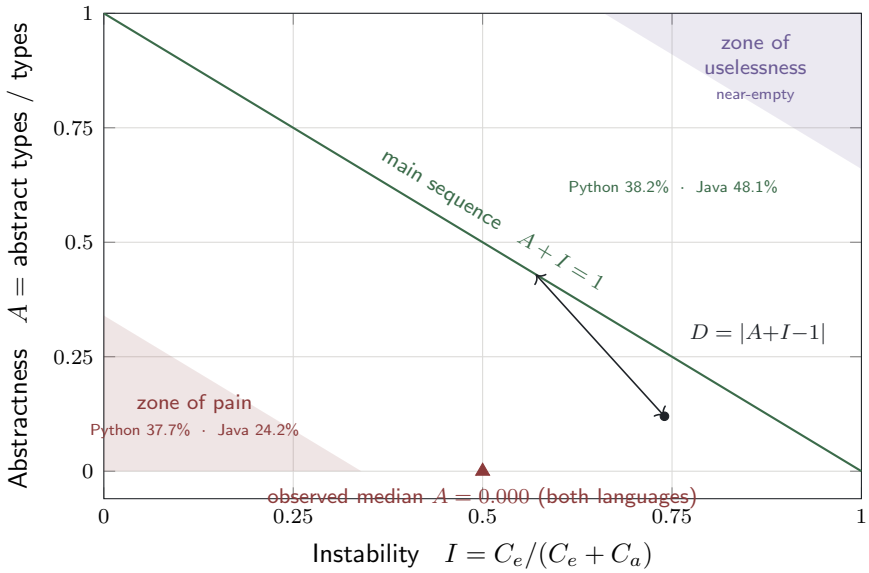


Figure 13: Package position, after Martin. A package that is concrete and heavily depended upon (bottom left) is painful to change; one that is abstract and depended upon by nothing (top right) is dead weight. D measures distance from the balanced diagonal. The median package abstractness in this corpus is **0.000** in both languages — half of all packages contain no abstract type at all, which is the strongest package-level statement the data supports. Position is reported as a descriptor only: 48.2% of Python packages are unplaceable because they contain no classes, against 5.7% in Java, so the measurement has a precondition that one language frequently fails to meet.

14. Validation protocol

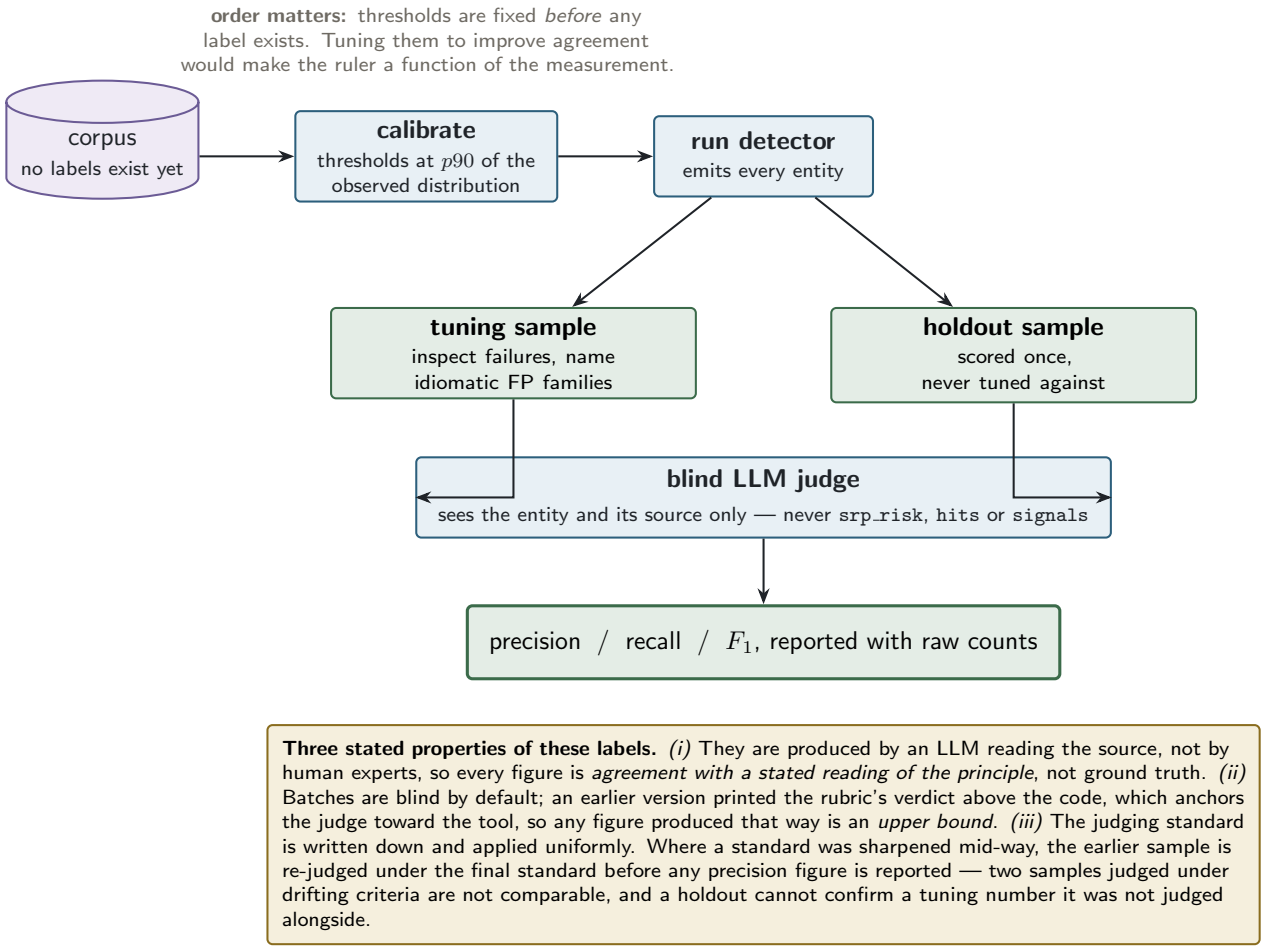


Figure 14: Validation protocol. Two structural properties defend against circularity: thresholds are derived from the corpus distribution before labelling begins, and the judge never sees the tool's own verdict. The holdout is scored once — once it has informed a design decision it is spent, and a further change requires a fresh sample.

Figure sources. All counts are from the 2026-08 run on `AIDev-7.6M` (snapshot cutoff 2026-03-31): 1 464 analyzable Python pull requests across 484 repositories and 13 310 source files, drawn from three agents at a 500-PR cap. Validation figures are reported separately per principle and are not pooled across principles, since the detectors differ in both unit of analysis and precision.